

Hayden Fuller
Modern Communication Systems F25
Lab 2
09/29/25

Matlab Simulations:

1)

The simulated BER curve closely follows the analytical $0.5 \cdot \text{erfc}(\sqrt{E_b/N_0})$ curve across the SNR range, confirming the Monte Carlo simulation is implemented correctly. At low SNR the simulation and theory match well because many errors yield good statistical averaging; at high SNR you may see a bit more Monte Carlo variability since errors become rare and the finite `num_bits = 1e5` limits accuracy. Slight discrepancies (tiny offsets) are expected from finite-sample randomness and the particular seed.

2)

- We reduced the per-sample noise variance by a factor L , which models a processing gain, more samples per symbol (or effective integration) spreads the noise so that per-sample variance is smaller for the same total bit energy.
- Because detection uses the sign of the summed metric, lowering per-sample variance reduces the probability that noise flips the sign, hence lower BER.
- This is a simplified model, in a full matched-filter / sampling-rate model the overall noise-energy per symbol is preserved and the improvement depends on filtering and sampling strategy; the current approach illustrates the idea of processing gain from oversampling/integration.

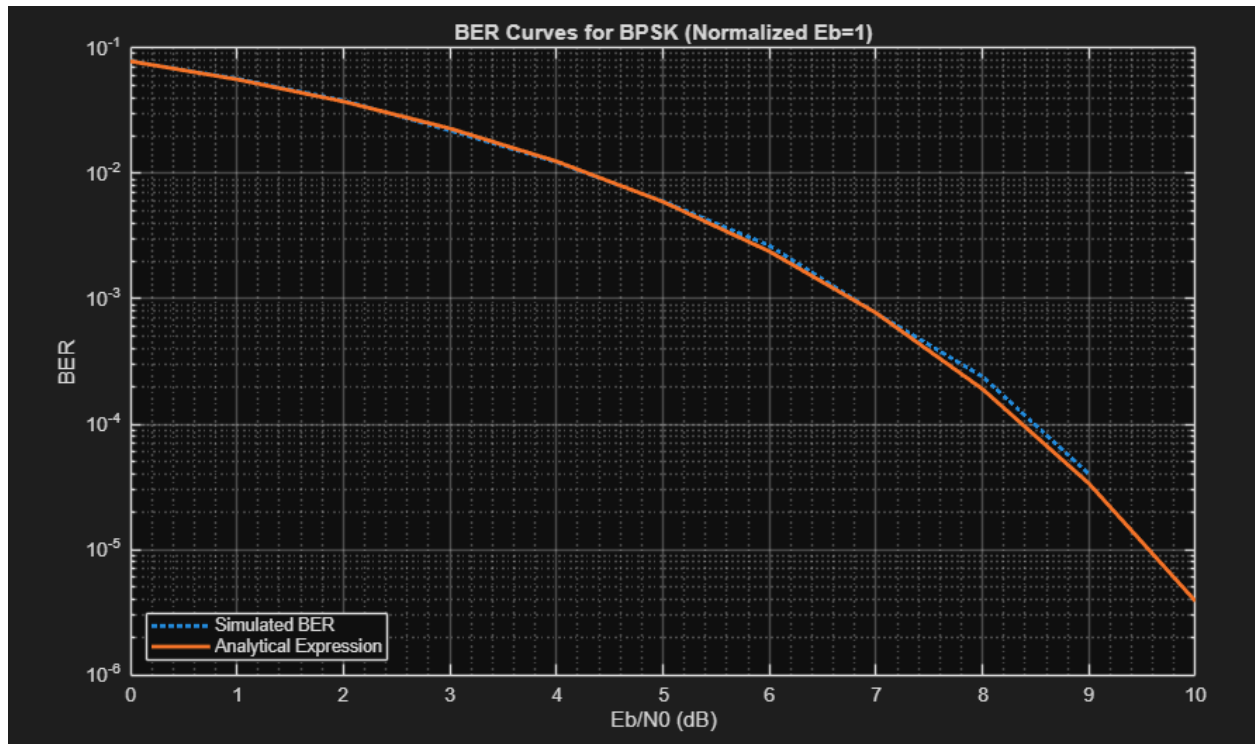


Figure 1

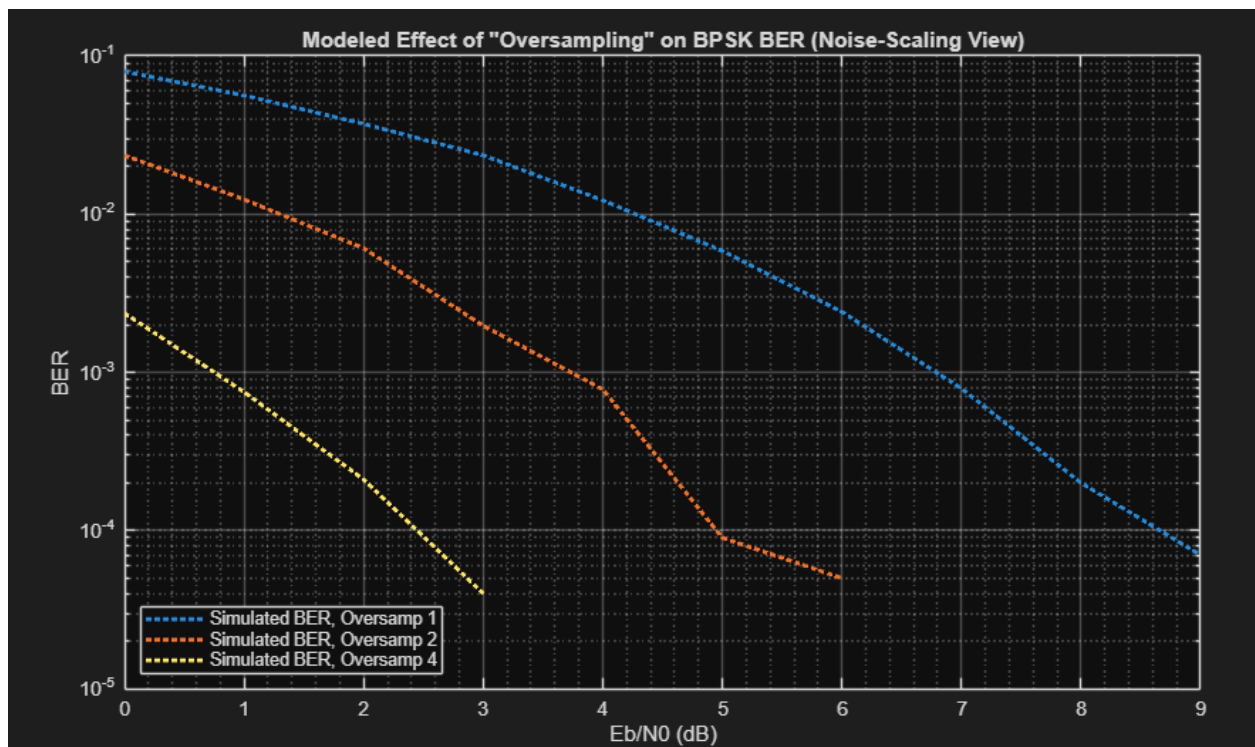


Figure 2

```

%% BPSK BER Simulation
% -----
clear; clc;
rng(123); % for Reproducibility
%% ----- Simulation Parameters -----
num_bits = 1e5; % Number of BPSK symbols (bits) to transmit in each trial
snr_db_vec = 0:1:10; % Vector of Eb/N0 points in dB to simulate
num_snrs = numel(snr_db_vec);
% Generate random BPSK symbols in {-1, +1}
bits = randi([0 1], 1, num_bits); % bits = 0/1
bpsk_syms = 2*bits - 1; % map to -1/+1
%% ----- Part 1: Single "oversampling" setting -----
% We simulate BER for a single oversampling factor (oversamp_fac = 1).
% With oversamp_fac = 1, the "effective" noise variance equals the baseline.
oversamp_fac = 1; % No oversampling-based noise scaling
ber_vec = zeros(1, num_snrs); % Preallocate simulated BER array
Eb = 1; % normalized
for i = 1:num_snrs
    % Select Eb/N0 for this run from snr_db_vec, db to linear scale mapping
    snr_db = snr_db_vec(i);
    ebn0 = 10^(snr_db/10); % linear Eb/N0
    % Energy per bit (normalized): Eb = 1 (dimensionless)
    % Under this normalization, N0 = Eb / (Eb/N0).
    n0 = Eb / ebn0;
    % Effective noise scaling due to oversampling:
    n0_effective = n0 / oversamp_fac;
    % Generate real AWGN samples: variance = (n0_effective)/2
    variance = n0_effective/2;
    sigma = sqrt(variance);
    noise = sigma * randn(1, num_bits);
    % Pass bpsk_syms through AWGN channel to get noisy symbols
    y = bpsk_syms + noise;
    % Hard-decision detection for BPSK:
    rx = ones(size(y));
    rx(y < 0) = -1;
    % Compute BER:
    num_errors = sum(abs(rx - bpsk_syms)/2);
    ber_vec(i) = num_errors / num_bits;
end
% Analytical BER for coherent BPSK in AWGN:
%  $P_b = Q(\sqrt{2E_b/N_0}) = 0.5 \cdot \text{erfc}(\sqrt{E_b/N_0})$ .
ebn0_vec = 10.^(snr_db_vec/10);
ber_vec_analytical = 0.5 * erfc( sqrt(ebn0_vec) );
% Plot (Part 1): Simulated vs Analytical
figure(1); clf;
semilogy(snr_db_vec, ber_vec, 'LineWidth', 2, 'LineStyle', ':'); hold on;
semilogy(snr_db_vec, ber_vec_analytical, 'LineWidth', 2);
legend('Simulated BER', 'Analytical Expression', 'Location', 'southwest');

```

```

title('BER Curves for BPSK (Normalized Eb=1)');
xlabel('Eb/N0 (dB)');
ylabel('BER');
grid on; grid minor;
%% ----- Part 2: "Oversampling" sweep via effective-noise scaling -----
oversamp_fac_vec = [1, 2, 4]; % Oversampling-like scaling values
ber_mat = zeros(numel(oversamp_fac_vec), num_snrs); % BER for each oversamp + SNR
for osamp_idx = 1:numel(oversamp_fac_vec)
    oversamp_fac = oversamp_fac_vec(osamp_idx);
    for i = 1:num_snrs
        snr_db = snr_db_vec(i);
        ebn0 = 10^(snr_db/10);
        n0 = Eb / ebn0;
        n0_effective = n0 / oversamp_fac; % reduce per-sample noise variance
        variance = n0_effective / 2;
        sigma = sqrt(variance);
        noise = sigma * randn(1, num_bits);
        y = bpsk_syms + noise;
        rx = ones(size(y));
        rx(y < 0) = -1;
        num_errors = sum(abs(rx - bpsk_syms)/2);
        ber_mat(osamp_idx, i) = num_errors / num_bits;
    end
end
% Plot (Part 2): Effect of "oversampling" (via noise scaling) on BER
figure(2); clf;
semilogy(snr_db_vec, ber_mat(1,:), 'LineWidth', 2, 'LineStyle', ':'); hold on;
semilogy(snr_db_vec, ber_mat(2,:), 'LineWidth', 2, 'LineStyle', ':');
semilogy(snr_db_vec, ber_mat(3,:), 'LineWidth', 2, 'LineStyle', ':');
legend('Simulated BER, Oversamp 1', ...
       'Simulated BER, Oversamp 2', ...
       'Simulated BER, Oversamp 4', ...
       'Location', 'southwest');
xlabel('Eb/N0 (dB)');
ylabel('BER');
title('Modeled Effect of "Oversampling" on BPSK BER (Noise-Scaling View)');
grid on; grid minor;

```